

ECE 411: mp_ooo Final Report

A 2-Way Superscalar Out-of-Order RV32IM Processor with Explicit Register Renaming

Ashmit Dutta

Luke Smith

David Mun

Team: VadaPav

University of Illinois at Urbana-Champaign

May 22, 2026

Abstract

This report describes the design, implementation, and evaluation of an out-of-order RV32IM processor targeting the ECE 411 mp_ooo specification. Our processor implements an Explicit Register Renaming (ERR) microarchitecture with split L1 instruction and data caches backed by a banked-burst DRAM model. We summarize the architecture, present the advanced features we implemented, and quantify their impact on IPC and PD^4 across the six staff-provided benchmarks.

Our final submitted processor for the class competition averages an IPC of 0.77 across the six CP3 benchmarks, with a peak of 1.06 on **compression**, and achieves a geometric-mean PD^4 of approximately 253 (with area penalty)—a $3.6\times$ improvement over the staff baseline. These marks were achieved through a 2-way superscalar execution, and a tournament branch predictor with branch target buffer with return address stack. With a memory-unit split LSQ, we reach a peak of 1.325 IPC on **compression**.

Contents

1	Introduction	3
2	Project Overview	3
2.1	Team Organization	3
2.2	Notable Achievements	4
3	Design Description	5
3.1	Overview	5
3.1.1	Front End	6
3.1.2	Backend	6
3.2	Milestones	7
3.2.1	Checkpoint 1	7
3.2.2	Checkpoint 2	8
3.2.3	Checkpoint 3	10
3.3	Advanced Design Options	11
3.3.1	2-way Superscalar	11
3.3.2	Split LSQ	12
3.3.3	Branch Prediction: Tournament Predictor, BTB, and RAS	14
4	Additional Observations	16
4.1	Methodology	16
4.2	Performance Counter Analysis: CoreMark IM	16
4.3	Final Results vs. Baseline	17
5	Conclusion	17

1 Introduction

The objective of `mp_000` is to design a synthesizable out-of-order processor implementing the RV32IM ISA, excluding `FENCE*`, `ECALL`, `EBREAK`, and `CSRR` instructions. Our CPU uses explicit register renaming with Tomasulo-style dynamic scheduling. What this means is that our processor uses typical register renaming structures (RAT, RRF, free list, and PRF) and our reservation stations carry physical register indices and ready bits. Operands are then pulled from the PRF at issue, not from the reservation stations themselves. We kept this design this way throughout the project (starting from CP2) because it was closer to how modern out of order processors do it. More specifically, this design decision also had a simpler recovery path for flushing and we had more flexibility to add ALUs and other modules without greatly affecting the downstream datapaths.

In embedded systems and in computer design workflows, this project served as a needed learning experience. The design competition added an urgency to not only implement what was explicitly needed for the project, but to also spend time optimizing and truly understanding the implications of the choices our group made. The remainder of this report is organized as follows:

- Section 2 presents the project goals and team organization.
- Section 3 describes the full design, covering the baseline microarchitecture, our incremental progress through three checkpoints, and our advanced design features with quantitative analysis.
- Section 4 presents additional performance observations and final results against the staff baseline.
- Section 5 summarizes lessons learned.

2 Project Overview

We framed our high-level objectives around the course design competition alongside the minimum checkpoint specifications. This is what motivated most of the design, since we knew that early on we would need the flexibility to implement the functionality that we need without restructuring core datapaths. Concretely, we parameterized the sizes of the ROB, physical register file, and branch predictor tables in `types.sv` and in some of our modules.

The flexibility-first decision was the most consequential of our project at a high level. The tournament predictor's banked storage, the RAS and BTB, the split load/store queue, and the second issue lane were all either added or upgraded after CP3, and they integrated cleanly only because the early-checkpoint datapaths had not been hard-coded to a single configuration.

2.1 Team Organization

On the advice of our mentor TA, we did not partition the project along frontend/backend lines. Instead, we split work along functional axes: one member owned a specific datapath function end-to-end so that every member stayed fluent across the full pipeline. This avoided the integration cliff that typically appears when the independently developed frontend and backend halves first

meet. We also collaborated on any decision that crossed the module boundaries, keeping all three members able to debug any part of the codebase.

2.2 Notable Achievements

Our processor currently ranks in the top 10 of the course design competition leaderboard. The design passes lint and synthesis without warnings, and matches Spike and RVFI on every released benchmark.

3 Design Description

3.1 Overview

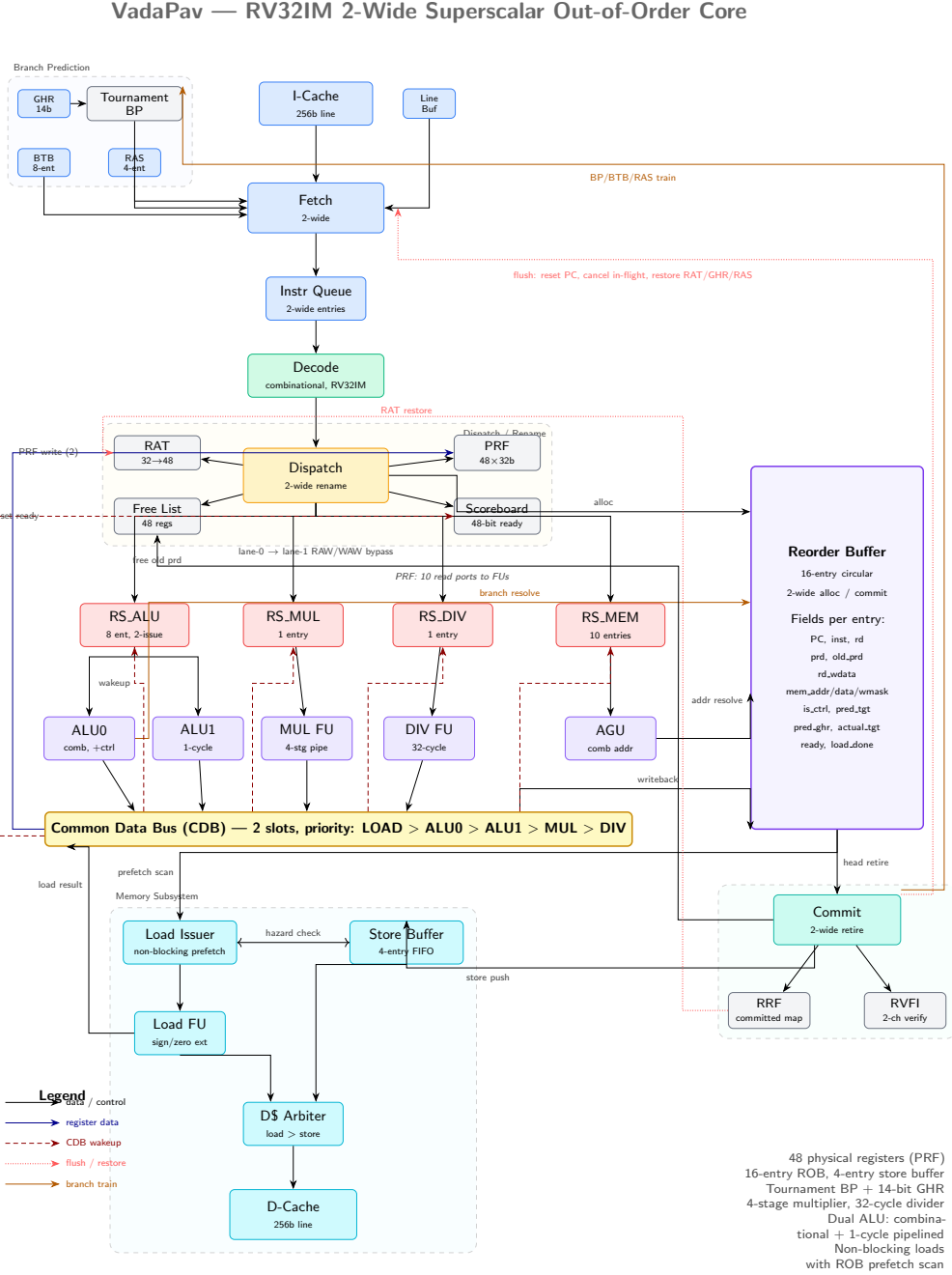


Figure 1: Top-level datapath block diagram. Solid lines indicate data flow; dashed lines indicate control/wakeup signals.

3.1.1 Front End

The front end consists of the program counter, an instruction cache fronted by a 256-bit linebuffer, and a decode/dispatch stage that inserts micro-ops into the issue queue and reorder buffer (ROB). The linebuffer eliminates the every-other-cycle stall imposed by `mp_cache`'s response timing, allowing 1 IPC fetch on sequential accesses within a cacheline. We later parallelized `fetch.sv` to be 2-wide, giving a primary and victim linebuffer that was checked in parallel every cycle.

Another parallelized process was `decode.sv`, the stateless stage between the instruction queue and the backend. `dispatch.sv` then performs the function unit classifications with free list, allocation, register renaming, and source readiness combinationally in the same clock cycle. For branch handling, we initially had a “predict not taken” implementation for CP3, but this was changed to a tournament style branch predictor that combines a bimodal table and a gshare table with a chooser counter that would only train when the bimodal and gshare predictions disagree.

One frontend bottleneck was in our branch target buffer. The single query port performance is under-served in comparison to a more parallelized approach with either multiple BTBs or a 2-way query port for one module.

3.1.2 Backend

The backend consists of our register renaming system, dispatch, reservation stations, ALUs, ROB, commit, common data bus, and recovery datapaths.

At any given cycle, dispatch presents two source register pairs to the RAT (32-entry table), which returns the current physical register for each source. The free list handles destination registers using our 48-entry PRF. The advantage of using this scheme is that only true RAW dependencies need to be accounted for. Dispatch is the gate for this.

Dispatch checks that the ROB has room, the free list contains physical registers, and the target reservation station has room and does not contain a conflict. Dispatch then pops the free list, allocates the ROB, and inserts into the RS and then snoops the CDB to set each source's ready bits. Without this snoop, the wakeup would arrive one cycle before the instruction was visible to the RS.

We have four reservation stations, each corresponding to the different ALU types (ALU, MUL, DIV, and MEM). Each RS entry receives a wakeup broadcast from the CDB that snoops the physical registers. If there is a match, the corresponding RS entry flips a ready bit so it can be issued to a functional unit. Executed instructions write back their destination physical register, ROB index, and result metadata through the CDB and ROB update paths.

The ROB is a 16-entry circular queue that turns our out-of-order execution into an in-order commit. Each entry records the original instruction, predicted vs. actual control flow, memory metadata for loads/stores, and the `rd_old_phys` value for free-list recycling. All data speculation lives here, so the ROB is responsible for branch writebacks, AGU writebacks, and the CDB writebacks. The ROB head controls when commit fires and retires up to 2 entries from the queue. Commit also pushes `rd_old_phys` back to the free list. Every commit updates the RRF table, which is a functional copy of the RAT that is used to rollback system state on a branch mispredict.

When the ALU resolves a control instruction and there is a mispredict, at commit time the ROB asserts a flush. Flush bulk- copies the RRF into the RAT in one cycle, clears the ROB/RS/free-list

around the queue causing the low pointer bits to match. Overall, our queue supported enqueue, dequeue, full, empty, reset, wraparound, and backpressure behavior when full to ensure that fetches stalled rather than overwriting unread instructions.

A key feature in CP1 was the linebuffer. It stored a 256-bit cacheline, which contained eight RV32 instructions. On a linebuffer hit, fetch would select the requested word using `pc[4:2]` and enqueue one instruction per cycle from the cacheline. However, on a miss, fetch would issue a cacheline-aligned request on the I-cache and wait for the returned line. This avoided the old `mp_cache` limitation where consecutive hits could still be limited to 0.5 IPC.

CP1 also introduced the cacheline adapter that allowed for the 64-bit DRAM bursts to fit a 256-bit interface. For reads, it would take four 64-bit bursts and store them into a single 256-bit `dfp_rdata` value and raise the `dfp_resp` when the line was full. For writes, we serialized the 256-bit line into four 64-bit bursts.

To test our CP1, we focused on FIFO edge cases, linebuffer hit and miss behavior, cacheline word selection, advancing the PC on successful enqueues, ensuring cacheline requests were aligned, and verifying that our adapter was correctly bursting reads and writes with correct ordering.

3.2.2 Checkpoint 2

For checkpoint 2, we focused on creating an out-of-order execution backend to support immediate and register-register ALU instructions, multiply, divide, and remainder instructions. In order to do this, we had to extend the decode and build a physical register file, a RAT, a free list, reservation stations, a reorder buffer, and a common data bus.

When building our backend, we created 64 entries in the physical register file and 32 entries in the ROB. We selected these sizes so we could exploit out-of-order behavior during future checkpoints. However, as we further developed our processor, we changed several of these values for timing, area, and power needs. Therefore, Table 1 shows the final optimized values used by the completed processor. The RAT mapped architectural registers to physical registers, and the retirement register file tracked the committed architectural mapping. The dispatch managed destination physical registers, checked the physical register file to see if sources were ready, and monitored the CDB for producer wakeups. Additionally, if any of the stations or tables were full, dispatch was able to apply backpressure.

Moving to the reservation stations, each of our reservation stations stored the ROB index, destination physical register, source physical tags, readiness bits, and immediate and control metadata. When both source operands became ready, the RS would move the instruction to its corresponding functional unit. After completion, the CDB selected one completed result per cycle while prioritizing the ALU output, and broadcast the destination physical tag, result value, and ROB index. This would subsequently wake up the dependent RS entries, update the physical register file, and mark the ROB entry as ready.

Finally, in order to maintain in-order commits, we built the ROB. Whenever instructions at the head of the ROB were completed, commit updated the RRF, and returned the old physical register to the free list.

As we were building each component, we created small testbenches to verify them. Then at the end of CP2, we created targeted programs such as `cp2_alu.basic.s`, `cp2_hazard_test.s`,

`cp2_mul_test.s`, `cp2_div_test.s`, `cp2_stress.s`, and `ooo_test.s`. These tests would check the rename path, RAW dependency wakeup, multiply and divide execution, CDB arbitration, out-of-order execution of nonconflicting instructions, and in-order retirement.

Table 1: Microarchitectural structure sizes

Structure	Size	Description / Rationale
I-cache	3-way, 64 sets, 32B lines (6 KB)	32B lines matches the width of a DRAM burst. 4-way I-caches are typical, but we were initially failing area, so we reduced to a 3-way associativity to improve the hit rate while incrementally decreasing power and area.
Linebuffer	2 slots $\times$ 32 B	Our linebuffer had 1 slot originally, but it was later changed with the implementation of superscalar. 32B is per spec from our 256 bit register. SRAM access is gated when either linebuffer slot hits or when IQ is full (<code>ufp_rmask = 0</code>), eliminating wasted I-cache reads on the common path.
IQ	12 entries, 2-enq/2-deq	Sized to ride out short rename/free-list/RS pressure without slowing down the linebuffer. The instruction queue is flushed with a specialized flush reset signal, so no per-entry kill bit was implemented.
BTB	8 entries	Sized with area and power in mind. Increasing the size of BTB didn't actually increase our performance much since it targets indirect <code>JALR</code> instructions only.
RAS	4 entries	Sized to cover the depth observed in CoreMark/Dhrystone-like benchmarks; deeper stacks pay area without much return on investment.
RAT	32 entries $\times$ 6-bit <code>phys_idx</code>	Sized to fit the RV32I spec. The RAT bulk single-cycle restores from the RRF snapshot on a flush, so both are sized the same way.
Free list	16 free physical registers (<code>NUM_PHYS_REG - 32</code>)	Bitmap representation so the two lowest-numbered free regs can be allocated combinationally each cycle and frees can land anywhere. The bitmap is the complement of the committed RAT image.
PRF	48 physical registers	Started speculating at 64 for a wider window then dropped to 48 once area pressure showed the upper 16 registers were rarely held. Our window roughly matches the 16-entry ROB.
ROB	16 entries (power of 2)	Below 32 by design since a wider ROB hit area/timing without proportional IPC. CDB contention became the dominant stall first so ROB was changed.
ALU RS	8 entries	Sized down from 12 to 8 once <code>perf_counters</code> showed <code>rs_alu_full</code> was a minor stall contributor. The reclaimed area went to D-cache and PRF.
MUL RS	1 entry	A deeper MUL RS was not justifiable. We made it 1 entry and that never changed.
DIV RS	1 entry	Similar to MUL RS, we could not justify changing DIV RS depth for the same reason.

continued on next page

Table 1 – *continued from previous page*

Structure	Size	Description / Rationale
MEM RS	10 entries	Largest RS in the design. We needed to accommodate many loads for benchmarks like mergesort. Stores also wait until commit to read store-data from PRF, so more entries were needed.
CDB	2 slots	This was 1 slot until we integrated advanced features; 2 slots match the 2-wide commit / 2-wide ROB writeback / 2 PRF write ports.
D-cache	4-way, 16 sets, 32B lines (2KB)	Like I-cache, 32B lines match DRAM burst geometry. Keeping the way count as-is ended up affecting area/power less than the I-cache did.

3.2.3 Checkpoint 3

For checkpoint 3, we completed the baseline RV32IM processor by adding memory instructions, control-flow handling, D-cache integration, and I-cache and D-cache arbitration. At this stage, the processor supported the full required RV32IM subset, excluding only `FENCE*`, `ECALL`, `EBREAK`, and `CSR` as allowed by the specification.

For the memory path, we added a MEM reservation station and an address generation unit. Loads and stores were renamed and scheduled like the other instructions, but instead of producing a normal ALU result, the AGU produced the resolved memory address and memory metadata. The AGU computed the effective address using `rs1 + imm`. For stores, it also generated the byte write mask and shifted store data into the correct byte lane. For loads, the processor supported `lb`, `lh`, `lw`, `lbu`, and `lhu` by selecting the correct byte or halfword from the cache response and applying either sign extension or zero extension. For stores, the processor supported `sb`, `sh`, and `sw` using the generated byte masks and shifted write data.

To keep memory execution correct, we used a conservative commit-time memory policy. Loads and stores were tracked in the ROB with their resolved address, write data, and write mask. When a load reached commit, the commit FSM issued a D-cache read, waited for the response, and wrote the loaded value back through the load side of the CDB. When a store reached commit, the FSM issued the D-cache write and retired the store after the cache acknowledged it. This was slower than allowing more aggressive memory execution, but it kept stores safe because speculative stores never updated the cache.

CP3 also added the data cache and memory arbitration logic. The D-cache reused the cacheline adapter path, so we extended the adapter to support both read and write bursts. Since the DRAM only had one port, we added an arbiter between the I-cache and D-cache so instruction fetches and data memory operations could share the same DRAM-facing interface.

For control flow, we added support for branches, `JAL`, `JALR`, and `AUIPC`. Conditional branches used comparison logic and a PC-plus-immediate target calculation. The ROB tracked whether a control instruction was mispredicted and stored the correct redirect target. When a mispredicted branch or jump reached the head of the ROB, commit triggered a flush. The flush redirected fetch, cleared the instruction queue, ROB, and reservation stations, and restored the RAT from

the committed RRF mapping. This allowed the processor to recover from wrong-path instructions while keeping the architectural state correct.

To test CP3, we added directed assembly programs for branch basics, JAL/JALR, flush stress, memory basics, subword memory operations, and memory hazards. These tests checked the AGU, D-cache path, I-cache/D-cache arbitration, commit-time memory FSM, and branch recovery logic. Additionally, we had to pass lint, pass synthesis, and run the released benchmarks. These benchmarks included Coremark IM, FFT, AES-SHA, Image, Compression, and Mergesort.

3.3 Advanced Design Options

Our advanced features span four categories for a total of 23 points:

- **Adv. Microarch:** 2-way Superscalar (10 pts)
- **Memory Unit:** Split LSQ with OoO loads (6 pts)
- **Branch Predictors:** GShare + Bimodal + Tournament + BTB (5 pts, FF-based)
- **Misc:** RAS (2 pts)

3.3.1 2-way Superscalar

Design The 2-way superscalar extension widened the processor front-to-back to sustain two instructions per cycle. In the frontend, `fetch.sv` was parallelized with a primary and victim linebuffer, each holding one 256-bit cacheline. On a hit, fetch extracts two aligned instructions per cycle; on a miss the second slot stalls until the cacheline returns. `decode.sv` was similarly widened to produce two micro-ops per cycle for dispatch.

Dispatch allocates two ROB entries, two physical destination registers from the free list, and two reservation station slots in a single cycle when resources are available. If only one slot can be allocated (e.g. the free list has one register remaining), dispatch degrades to single-issue for that cycle. The CDB was widened to two slots with fixed priority: slot 0 serves the ALU and slot 1 serves the multiplier, divider, or memory unit, avoiding arbitration logic at the cost of occasional slot waste.

Commit retires up to two ROB entries per cycle in program order. The second retire is suppressed if the ROB head triggers a flush or if the second entry is not yet ready. The instruction queue between fetch and decode supports 2-enqueue and 2-dequeue per cycle, sized at 10 entries to absorb fetch/decode rate mismatches across cache-miss bubbles.

The main trade-off is area and power: widening dispatch, CDB, and commit roughly doubles the wiring and mux fanout in those stages, increasing dynamic power. On workloads with limited ILP or frequent stalls, the second slot is often unused—the commit-width histogram showed only 12% of cycles committing two instructions on `coremark_im`—but the area cost is paid regardless.

Testing Correctness was verified through RVFI checking against Spike on all six CP3 benchmarks. The key concern for 2-wide execution is that the second instruction might observe stale state or

miss a dependency on the first instruction dispatched in the same cycle. We verified this with directed tests placing dependent instruction pairs back-to-back (e.g. `add x1, x2, x3` followed by `add x4, x1, x5`) and confirmed that dispatch correctly detects the RAW hazard and either forwards the physical register tag or serializes the pair. The full benchmark suite passing with zero RVFI mismatches confirmed 2-wide correctness under realistic workloads.

Performance Analysis Table 2 shows the estimated impact of 2-way superscalar execution. The “Before” IPC was back-calculated from the commit-width histogram: in a hypothetical 1-wide processor, cycles that committed two instructions would each require two cycles, while all other cycles remain unchanged. `compression` benefits the most (+35%) because its tight computational loops expose significant instruction-level parallelism, pushing IPC from 0.776 to 1.045. `fft` (+28%) and `mergesort` (+22%) also show strong gains from their regular access patterns and loop-carried parallelism. `aes_sha` benefits least (+6%) because its 28.5% branch misprediction rate causes frequent pipeline flushes that waste the second issue slot. The PD^4 improvement is amplified by the quartic delay exponent: even a modest IPC increase translates into a large reduction in the delay⁴ component.

Table 2: Impact of 2-way Superscalar.

Benchmark	IPC		PD^4	
	Before	After	Before	After
coremark_im	0.610	0.684	44.78	28.35
fft	0.751	0.962	4868	1811
aes_sha	0.340	0.359	22700	18256
image	0.344	0.380	2531	1698
compression	0.776	1.045	82.28	25.06
mergesort	0.607	0.740	342.4	155.3

3.3.2 Split LSQ

Design Before implementing the split LSQ, our original design implemented the load issue, store retirement, and committed-store draining using the ROB and a post-commit store buffer. Although this worked, it overloaded the ROB because it was in charge of numerous memory ordering decisions. This design also forced loads to wait behind stores that were not conflicting. To fix this, we replaced this design by adding a split load queue and store queue in `split_lsq.sv`. The LQ was in charge of tracking loads until they had a resolved address, issuing the loads to the data cache or retrieving forwarded data from the SQ, and retiring the loads in program order. The SQ was in charge of dispatching stores from allocation through address resolution, committing, draining to the data cache, and removing drained stores from the queue.

The split LSQ sits between the grouping of dispatch, ROB, and AGU and the existing `load_issuer` and `dcache_arb`. The dispatch first allocates the LQ/SQ entries and records the queue index in the ROB and memory reservation station. Later, when the AGU finishes, it fills in the load and store fields, such as the address, using the queue indexes. For each load that is ready, the LSQ scans older SQ entries. A conservative design decision that we decided to make

was if an older store has an unknown address, the load blocks. However, if all older stores are resolved and no word conflicts exist, the load can issue out of order with respect to stores at different addresses. Additionally, if the youngest matching older store fully covers the load’s byte mask, the LSQ forwards the value directly from the SQ instead of accessing the data cache. However, like the unresolved address, we decided to make partial byte overlaps stall until the store drains. In addition to the forwarding and out-of-order issuing, store draining is also owned by the SQ. When the ROB retires a store, it is marked as committed, but it stays in the SQ until after the data cache write is finished. This ensures that any extra forwarding or violation searching can be done up until the point a store is officially drained. The head committed store drains through the commit-side port of `dcache_arb` and is popped only after a drain response. Finally, to avoid starving stores when running programs that have many loads, the SQ uses a commit-priority hint after four loads while a store is waiting. Overall, our design lets loads pass resolved non-conflicting stores, allows for store-to-load forwarding, ensures correctness, and handles load and store operations in a separate memory unit outside of the ROB logic, but it still stalls behind unknown older stores and partial overlaps.

Testing We tested the split LSQ with three batches of tests. First, we used the directed memory tests from CP3, `cp3_mem_basic.s`, `cp3_mem_subword.s`, `cp3_mem_hazard.s`, and `ss_mem_serialize.s`, to confirm that ordinary loads, stores, subword masks, and in-order memory serialization matched the RVFI reference model. Second, we added LSQ tests: `ss_fwd_full.s` checks full-word store-to-load forwarding, `ss_fwd_subword.s` checks byte and halfword forwarding, `ss_partial_stall.s` checks partial-overlap blocking, and `ss_drain_alias.s` checks that committed stores remain visible for forwarding. Finally, we ran all six release benchmarks with RVFI enabled and created multiple LSQ performance counters. For example, we added a `cnt_lsq_violation` counter, which counts the number of times a store saw that a younger load used an overlapping word. Our benchmark runs had zero violations. The counters also confirmed that forwarding was active, as we had 989 forwards on `coremark_im`, 4618 on `aes_sha`, 10251 on `fft`, 196 on `mergesort`, and 2119 on `image`.

Performance Analysis The split LSQ improves the memory pipeline by removing the old committed-store buffer and by allowing independent loads to pass while resolved stores wait in the SQ. Table 3 shows the post-LSQ IPC and PD^4 from our benchmark runs, and the “Before” columns use the run before implementing split LSQ at 500 MHz. IPC improved on all six benchmarks, while PD^4 improved on five of six benchmarks (`coremark_im`, `fft`, `aes_sha`, `image`, and `compression`), regressing only on `mergesort`. Split LSQ helped the most on memory-intensive workloads with numerous store-to-load forwarding opportunities. For example, `fft` forwarded 10251 loads, `aes_sha` forwarded 4618, and `image` forwarded 2119. However, it helped less on `compression`, where the forwarding counter was zero. This means that the workload was bottlenecked by another component outside of the LSQ. Finally, the unknown-store counter was zero on four benchmarks, 1.5% on `aes_sha`, and 2.2% on `fft`, which matches the expected behavior of allowing loads to pass resolved non-conflicting stores. Loads still need to wait when an older store address is not yet known.

Table 3: Impact of Split LSQ.

Benchmark	IPC		PD^4	
	Before	After	Before	After
coremark_im	0.8218	0.8517	15.07	13.057
fft	0.9532	0.9977	1956	1628.474
aes_sha	0.5786	0.6627	3209	1860.094
image	0.3971	0.4049	1784	1650.686
compression	1.0561	1.3251	21.17	8.545
mergesort	0.8337	0.7975	83.88	100.080

3.3.3 Branch Prediction: Tournament Predictor, BTB, and RAS

Design The frontend implements a McFarling-style tournament branch predictor [1] (`bp_tournament.sv`) paired with a Branch Target Buffer (`btb.sv`) and a Return Address Stack (`ras.sv`). The three together cover all control-flow types: conditional branches via the direction predictor, returns (JALR `x0`, `ra/t0`) via the RAS, and the remaining indirect jumps via the BTB. Unconditional JAL is handled statically by fetch. **All predictor tables are implemented with flip-flop arrays rather than SRAM macros, which simplifies the multi-ported read/write interface at the cost of higher area.**

Three 2-bit-counter tables are indexed by a folded-XOR hash of the pc and a pc_index in `bp_tournament.sv`: a PC-indexed bimodal table, a gshare table whose index is `pc_index` $\oplus$ GHR, and a chooser table that selects between them per PC. The implementation uses 256 entries per table. The bimodal counter and chooser counter share a PC index, so they are merged into a single 4-bit array (`bc_bank`, and each table is split into four banks of $N/4$ entries. Inactive bank offsets are gated to zero, so only one of four bank mux trees toggles per read — roughly a 75% reduction in read-side switching activity, which is the dominant power term in the predictor at our area. This was something we revisited many times, since the branch predictor ended up causing the majority of our power bottlenecks towards the end of the mp.

The BTB (`btb.sv`) is a direct-mapped 8-entry table tagged on the current pc. It is queried combinationally on the lane-0 PC and trained synchronously at commit, but only on committed taken JALR instructions that are *not* returns. Excluding returns is critical: a return target depends on caller, so caching it in a PC-indexed BTB would mispredict on every alternate caller. Returns are routed to the RAS instead.

The return address stack (`ras.sv`) keeps two stack pointers, one used at fetch and on that’s architectural and updated at commit. Calls (JAL/JALR `rd=ra/t0`) push `pc+4` and returns (JALR `rs1=ra/t0`, `rd!=ra/t0`) pop. On a flush, the fetch pointer snaps back to the committed one, restoring stack depth. The stack is instantiated at depth 4 (`cpu.sv`), which is a notable area savings but is shallow for heavily nested call chain code.

Trade-offs:

- *Tournament over plain gshare*: Gshare alone underperforms on PC-local patterns where short

global history aliases unrelated branches according to McFarling’s paper on the subject [1]. The chooser recovers those cases by deferring to bimodal, which performs better for high locality.

- *Merged bimodal+chooser array*: They share a PC index, so a single read port serves both. Costs one extra bit per entry but removes a separate table and its decoder. This was a compromise we made when getting the area down for this module.
- *Small BTB / shallow RAS*: 8 BTB entries and 4 RAS frames are sized for area; deeper structures would help JALR accuracy on benchmarks with many indirect call sites or deep recursion.

Testing Correctness was verified at two levels.

Unit-level: `bp_tournament`, `bp_ghr`, `btb`, and `ras` were each exercised with directed SystemVerilog testbenches.

Integration: The full processor was run under `top_tb` against Spike and the RVFI monitor. Any direction misprediction, target misprediction, or stale GHR/RAS state surfaces as either an RVFI mismatch or a Spike divergence, since the architectural state at commit must match Spike regardless of how speculation went. This was the primary correctness signal: zero Spike/RVFI mismatches across all six CP3 benchmarks indicated that our processor worked as expected. Fine tuning the IPC, area, and power ended up exposing small performance bugs.

Performance counters. `perf_counters.sv` exposes `cnt_branches_resolved`, `cnt_branches_mispredicted`, and a per-opcode misprediction breakdown (`cnt_mispred_br/jal/jalr`) which we used to confirm the predictor was actually training rather than coincidentally agreeing with the program. Non-zero `cnt_mispred_br` together with a falling ratio over the run indicates active learning; a constant ratio would suggest a stuck counter or a stale GHR.

Performance Analysis Table 4 shows the impact of replacing the static predict-not-taken policy with the tournament branch predictor. The “Before” values estimate performance under predict-not-taken at the final structure sizes (PRF-48, ROB-16). The most dramatic improvement is on `aes_sha`, whose branch-heavy control flow suffered a 28.5% misprediction rate under predict-not-taken; the tournament predictor reduced this substantially, yielding a 66% IPC improvement and a $7.6\times$ reduction in PD^4 . `coremark_im` improved 19% in IPC due to its mix of loop branches that the bimodal component predicts well. Compute-dominated benchmarks like `fft` and `compression` showed minimal change because they have few branches and their misprediction rates were already near zero. The PD^4 improvement is consistently positive across all six benchmarks because the predictor reduces delay without meaningfully increasing power.

Table 4: Impact of Dynamic Branch Prediction.

Benchmark	IPC		PD^4	
	Before	After	Before	After
coremark_im	0.690	0.822	25.42	12.62
fft	0.947	0.953	1798	1754
aes_sha	0.349	0.579	19497	2577
image	0.389	0.397	1740	1602
compression	1.050	1.056	22.41	21.90
mergesort	0.810	0.834	99.90	88.89

4 Additional Observations

4.1 Methodology

We synthesized at 500 MHz (2.0 ns clock period), meeting timing with near-zero slack. The final design area is $304\,648\,\mu\text{m}^2$ as reported by Synopsys Design Compiler. Because the area exceeds the $300\,000\,\mu\text{m}^2$ threshold, an area penalty factor of $2^{(304,648-300,000)/50,000} \approx 1.066$ applies to all PD^4 values. Power was measured per benchmark via `scons power PROG=prog.elf` using SAIF-based switching activity, and delay was taken from *Monitor Segment Time* reported by `top_tb`, converted to ms. PD^4 is computed as $\text{Power (mW)} \times \text{Delay (ms)}^4$.

4.2 Performance Counter Analysis: CoreMark IM

To illustrate where cycles are spent in a representative workload, we profiled `coremark_im` using the hardware performance counters in `perf_counters.sv`. The run completed 304,044 instructions in 371,367 cycles at an IPC of 0.822.

Commit-width utilization. Of all cycles, 25.6% committed two instructions, 30.7% committed one, and 43.7% committed zero. The high fraction of idle commit cycles indicates that the pipeline is frequently starved. The ROB was empty for 33.6% of cycles, which points to frontend and branch-recovery bubbles rather than backend pressure. This is interesting because our D-Cache is only 2KB.

Partial-commit breakdown. When only one instruction committed, the dominant reason (59%) was that the second ROB entry was not yet ready (`psub_lane1_nrdy`), reflecting data-dependency chains that serialized execution. The head being a control instruction accounted for 16% of single-commit cycles (the ROB suppresses dual-commit when the head triggers a possible flush), and the second entry being a load or store accounted for another 13%.

Branch prediction. The tournament predictor resolved 60,475 branches with 4,497 mispredictions, a 7.4% overall misprediction rate. Breaking this down: conditional branches (`cnt_mispred_br`)

mispredicted at 6.3% (3,470 / 55,170), while JALR mispredicted at 41.2% (1,027 / 2,494). The high JALR rate reflects the shallow 8-entry BTB: indirect call sites that exceed the BTB capacity alias and evict each other. Each mispredict cost an average of $28,909/4,497 \approx 6.4$ drain cycles, consuming 7.8% of total execution time in post-flush recovery.

Fetch and I-cache. Fetch delivered two instructions in 74.3% of cycles, confirming strong linebuffer hit rates. The remaining 21.0% of zero-fetch cycles were almost entirely linebuffer misses (75,743 cycles) and post-flush drain (20,891 cycles). I-cache misses consumed 30,931 cycles (8.3%), while D-cache misses were a negligible 4,327 cycles (1.2%).

Structural hazards. The MEM reservation station was never full (0 cycles), validating that the split LSQ decoupled memory scheduling from the ROB effectively. The ALU RS was full for 5.2% of cycles, and the ROB was full for 11.5%. The instruction queue backed up for 10.1% of cycles, correlating with I-cache miss bubbles propagating downstream.

4.3 Final Results vs. Baseline

Table 5: Final processor metrics compared to the staff baseline. Bold indicates we beat the baseline on that metric.

Benchmark	Baseline			Ours		
	IPC	Power (mW)	PD^4	IPC	Power (mW)	PD^4
coremark_im	0.398	18.858	87.187	0.822	49.8	12.62
aes_sha	0.309	18.598	11826.027	0.579	49.8	2577
compression	0.484	18.917	188.395	1.056	49.8	21.90
fft	0.508	18.872	8229.957	0.953	49.8	1754
mergesort	0.431	18.865	471.743	0.834	49.8	88.89
image	0.376	18.821	750.796	0.397	49.8	1603

5 Conclusion

Wrapping up, our team built a working, out-of-order, RV32IM processor that is able to execute a variety of complex programs while maintaining correctness relative to RVFI and Spike verification. Our design uses explicit register renaming with a RAT, RRF, physical register file, free list, reservation stations, ROB, and a CDB wakeup to support speculative out-of-order execution while maintaining in-order retirement.

Across this project, we grew the processor from an instruction fetcher in CP1 into a complete processor equipped with caches, memory instructions, and control-flow recovery in CP3. In addition to this, we further improved our design by adding several advanced features, including a split load/store queue with forwarding, two-wide execution support, a tournament branch predictor, a BTB, and a return address stack.

We would say that the most novel part of our final processor is the two-wide superscalar design because it challenged us to redesign our fetch, decode, dispatch, and commit using the information

we learned about superscalar processors in lecture. In conclusion, our team did more than just build an out-of-order processor; we managed to reach 10th overall in the ECE 411 competition with some metrics even reaching 2nd place. This clearly highlights our team's hard work, our understanding of computer architecture, and our commitment to go above what was expected for us.

References

- [1] S. McFarling, "Combining branch predictors," Digital Equipment Corporation, Western Research Laboratory, Palo Alto, CA, Technical Note TN-36, Jun. 1993. [Online]. Available: <https://www.ece.ucdavis.edu/~akella/270W05/mcfarling93combining.pdf>